Code: https://github.com/zhurleonid/corporate-risk-management
(`homework_7/part2_evt_practice/hw7_part2_code.ipynb`)

Homework 7 comes as two problem sheets; this is sheet 2. The write-up is below and the code is appended at the end. The derivations of the four VaR formulas are handwritten and attached in front.

EVT risk measures for the S&P 500

Set-up

Daily S&P 500 closes from Yahoo Finance, 11/01/2011 to 12/28/2021 (`sp500_daily.csv`): $N + 1 = 2556$ closes, $N = 2555$ log returns. Yahoo has 2557 closes through 12/29/2021, so the end date is moved back one day as the sheet instructs.

$V = \$1{,}000{,}000$ each day, $L = -VX$. The α grid is $0.99 + 0.000099k$, $k = 1, \ldots, 100$: 100 values in increments of 0.000099, from 0.990099 to 0.9999.

Pseudocode

```
x_i = ln(S_i / S_{i-1}),  l_i = -V x_i,   i = 1 ... 2555
alpha_k = 0.99 + 0.000099 k,             k = 1 ... 100

(1) empirical
    VaR(a) = sorted l at position ceil(N a)

(2) normal
    mu, s2 = mean, variance of x_1 ... x_100
    for each later x_i up to 11/01/2021:
        s2 = theta s2 + (1 - theta)(x_i - mu)^2
        mu = lambda mu + (1 - lambda) x_i
    VaR(a) = -V mu + V sqrt(s2) Phi^{-1}(a)

(3) GEV, for (n, m) = (73, 35) and (35, 73)
    m_j = max of l over block j,  j = 1 ... m
    (xi, mu, sigma) = MLE GEV fit to m_1 ... m_m
    VaR(a) = mu + sigma/xi ((-n log a)^(-xi) - 1)

(4) GPD
    u = empirical VaR_0.95
    y = {l_i - u : l_i > u}
    (xi, beta) = MLE GPD fit to y, location 0
    VaR(a) = u + beta/xi ((0.05 / (1 - a))^xi - 1)
```

The fits use `scipy.stats.genextreme` (shape $c = -\xi$) and `scipy.stats.genpareto` (shape $c = \xi$) on losses in $ thousand; in dollars scipy's GEV optimizer stops at a wrong point.

(1) Empirical

> **By hand, attached:** $\mathrm{VaR}_\alpha = l_{(\lceil N\alpha \rceil)}$.

For $\alpha > 2554/2555 = 0.99961$ this is the largest loss in the sample, $127{,}652.20$ on 03/16/2020, so the curve is flat at the top of the grid.

(2) Normal with EWMA moments

> **By hand, attached:** $\mathrm{VaR}_\alpha = -V\mu_{t+\Delta} + V\sigma_{t+\Delta}\Phi^{-1}(\alpha)$.

EWMA started from the sample mean and variance of the first 100 returns and run through 11/01/2021 (2516 returns):

$$\mu_{t+\Delta} = 1.2101 \times 10^{-3}, \qquad \sigma_{t+\Delta} = 7.2271 \times 10^{-3}\ (11.47\%\ \text{annualized}).$$

(3) GEV, block maxima

> **By hand, attached:** $\mathrm{VaR}_\alpha = \mu + \dfrac{\sigma}{\xi}\Big((-n\log\alpha)^{-\xi} - 1\Big)$.

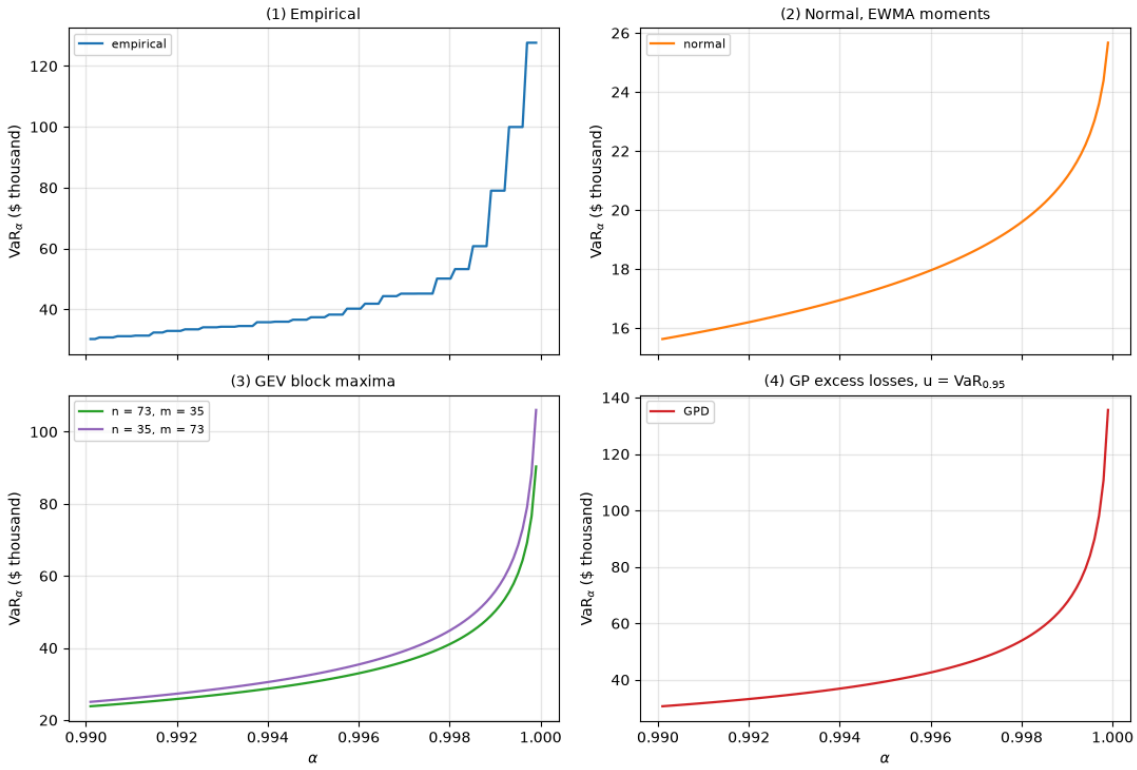
	ξ	μ	σ
$n = 73,\ m = 35$	0.1815	$20{,}958.16$	$8{,}732.71$
$n = 35,\ m = 73$	0.2141	$15{,}298.85$	$8{,}246.20$

(4) GP excess losses

> **By hand, attached:** $\mathrm{VaR}_\alpha = u + \dfrac{\beta}{\xi}\Big(\Big(\dfrac{1-F(u)}{1-\alpha}\Big)^{\xi} - 1\Big)$,
> $F(u) = 0.95$.

$u = \mathrm{VaR}_{0.95} = 15{,}252.86$, with 127 losses above it.

$$\xi = 0.2656, \qquad \beta = 7{,}594.30.$$



$\alpha \mapsto \mathrm{VaR}_\alpha$ *for the 100 values of* α *up to* 0.9999*, one panel per method.*

Results at $\alpha = 0.9999$

Method	$\mathrm{VaR}_{0.9999}$
(1) Empirical	127,652.20
(2) Normal, EWMA	25,667.50
(3) GEV, $n = 73$	90,352.51
(3) GEV, $n = 35$	106,042.15
(4) GPD	135,610.18

In dollars.

Highest: the GPD estimate, $135{,}610$. The empirical VaR is close behind but is capped at the largest observed loss. The normal VaR is about one fifth of the others.

GEV vs GPD: the difference is meaningful. The GPD VaR is 50% above the GEV with $n = 73$ and 28% above the GEV with $n = 35$, and the GEV estimate itself rises 17% when the block size goes from 73 to 35. At $\alpha = 0.9999$ the estimate is driven by the shape parameter, and the GEV fits rest on only 35 or 73 block maxima and give a smaller ξ (0.18, 0.21) than the GPD fit on 127 exceedances (0.27).

Appendix: source code

The code below produces every figure and number above.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm, genextreme, genpareto

V0 = 1_000_000
LAM = 0.97          # EWMA mean
THETA = 0.97        # EWMA variance
N_INIT = 100        # EWMA initialization period (days)
KD = 1000           # GEV / GPD fits are done in $ thousand

# 100 values of alpha in steps of 0.000099, ending at 0.9999 (sheet)
alphas = 0.99 + 0.000099 * np.arange(1, 101)

plt.rcParams['figure.figsize'] = (11, 5)


def var_emp(losses, a):
    # VaR from a sample: the ceil(n*a)-th order statistic
    ls = np.sort(np.asarray(losses))
    n = len(ls)
    return ls[int(np.ceil(n * a)) - 1]
```

In [2]:
```
# S&P 500 daily closes, saved to csv so this runs offline. To pull fresh dat
#     import yfinance as yf
#     yf.download('^GSPC', start='2011-11-01', end='2021-12-29')['Close']
# Yahoo has 2557 closes through 12/29/2021, one more than the sheet's
# N + 1 = 2556, so as the sheet says the end date is moved to 12/28/2021.
prices = pd.read_csv('sp500_daily.csv', parse_dates=['Date'])
prices = prices.set_index('Date')['SP500']

X = np.log(prices / prices.shift(1)).dropna()
x = X.values
N = len(x)
L = -V0 * x                          # linearized daily loss, $1M held each

print('N + 1 =', len(prices), 'closes,',
      prices.index[0].date(), '->', prices.index[-1].date())
print('N =', N, 'log returns')
print('alpha grid: %d values, %.6f to %.6f' % (len(alphas), alphas[0], alpha
print('largest loss %.2f on %s' % (L.max(), X.idxmin().date()))
```

```
N + 1 = 2556 closes, 2011-11-01 -> 2021-12-28
N = 2555 log returns
alpha grid: 100 values, 0.990099 to 0.999900
largest loss 127652.20 on 2020-03-16
```

In [3]:
```
var_1 = np.array([var_emp(L, a) for a in alphas])

# above alpha = (N-1)/N the order statistic is the largest loss
print('empirical VaR is the sample maximum for alpha > %.6f' % ((N - 1) / N)
print('VaR_0.9999 = %.2f' % var_1[-1])
```

```
empirical VaR is the sample maximum for alpha > 0.999609
VaR_0.9999 = 127652.20
```

In [4]:
```python
# initialize with the sample mean and variance of the first 100 returns,
# then run the recursions through the return on 11/01/2021
X_t = X.loc[:'2021-11-01']
x_t = X_t.values
mu = x_t[:N_INIT].mean()
s2 = x_t[:N_INIT].var(ddof=1)
for xi in x_t[N_INIT:]:
    s2 = THETA * s2 + (1 - THETA) * (xi - mu)**2      # uses the old mean
    mu = LAM * mu + (1 - LAM) * xi
sig = np.sqrt(s2)

print('returns used: %d (last %s)' % (len(x_t), X_t.index[-1].date()))
print('mu_{t+D} = %.6e   sigma_{t+D} = %.6e   (%.2f%% annualized)'
      % (mu, sig, 100 * sig * np.sqrt(252)))

# L = -V X ~ N(-V mu, V^2 sigma^2)
var_2 = -V0 * mu + V0 * sig * norm.ppf(alphas)
print('VaR_0.9999 = %.2f' % var_2[-1])
```

```
returns used: 2516 (last 2021-11-01)
mu_{t+D} = 1.210096e-03   sigma_{t+D} = 7.227070e-03   (11.47% annualized)
VaR_0.9999 = 25667.50
```

In [5]:
```python
def gev_var(xi, mu_g, sig_g, n, a):
    # VaR of one daily loss when the n-day block maximum ~ GEV(xi, mu, sigma
    return mu_g + sig_g / xi * ((-n * np.log(a))**(-xi) - 1)


gev = {}
for n in (73, 35):
    m = N // n
    maxima = L[:m * n].reshape(m, n).max(axis=1)        # block maxima of L
    c, loc, scale = genextreme.fit(maxima / KD)         # fit in $ thousand
    xi_g, mu_g, sig_g = -c, KD * loc, KD * scale        # back to dollars
    gev[n] = dict(m=m, xi=xi_g, mu=mu_g, sig=sig_g,
                  var=gev_var(xi_g, mu_g, sig_g, n, alphas))
    # check against scipy's own quantile: P[X <= x] = H(x)^(1/n)
    check = KD * genextreme.ppf(alphas**n, c, loc, scale)
    print('n = %d, m = %d blocks:  xi = %.4f   mu = %.2f   sigma = %.2f'
          % (n, m, xi_g, mu_g, sig_g))
    diff = np.abs(gev[n]['var'] - check).max()
    print('    VaR_0.9999 = %.2f   (max diff vs ppf %.1e)'
          % (gev[n]['var'][-1], diff))
```

```
n = 73, m = 35 blocks:  xi = 0.1815   mu = 20958.16   sigma = 8732.71
    VaR_0.9999 = 90352.51   (max diff vs ppf 5.8e-11)
n = 35, m = 73 blocks:  xi = 0.2141   mu = 15298.85   sigma = 8246.20
    VaR_0.9999 = 106042.15   (max diff vs ppf 2.0e-10)
```

In [6]:
```python
u = var_emp(L, 0.95)
excess = L[L > u] - u
F_u = 0.95                                        # by construction, as on the

# fit in $ thousand (location fixed at 0) and scale beta back to dollars
```

```
c, loc, scale = genpareto.fit(excess / KD, floc=0)
xi_p, beta = c, KD * scale                      # genpareto's c is xi itself

var_4 = u + beta / xi_p * (((1 - F_u) / (1 - alphas))**xi_p - 1)

print('u = VaR_0.95 = %.2f   exceedances: %d   (1 - N_u/N = %.5f)'
      % (u, len(excess), 1 - len(excess) / N))
print('xi = %.4f   beta = %.2f' % (xi_p, beta))
print('VaR_0.9999 = %.2f' % var_4[-1])
```

```
u = VaR_0.95 = 15252.86   exceedances: 127   (1 - N_u/N = 0.95029)
xi = 0.2656   beta = 7594.30
VaR_0.9999 = 135610.18
```

In [7]:

```
methods = {
    '(1) empirical': var_1,
    '(2) normal EWMA': var_2,
    '(3) GEV n=73': gev[73]['var'],
    '(3) GEV n=35': gev[35]['var'],
    '(4) GPD': var_4,
}
assert abs(alphas[-1] - 0.9999) < 1e-12
table = pd.Series({k: v[-1] for k, v in methods.items()}, name='VaR_0.9999')
print(table.round(2).to_string())
print('\nhighest:', table.idxmax())
print('GPD / GEV(73) = %.3f   GPD / GEV(35) = %.3f   GEV(73) / GEV(35) = %.3
      % (var_4[-1] / gev[73]['var'][-1], var_4[-1] / gev[35]['var'][-1],
         gev[73]['var'][-1] / gev[35]['var'][-1]))
```

```
(1) empirical      127652.20
(2) normal EWMA     25667.50
(3) GEV n=73        90352.51
(3) GEV n=35       106042.15
(4) GPD            135610.18

highest: (4) GPD
GPD / GEV(73) = 1.501   GPD / GEV(35) = 1.279   GEV(73) / GEV(35) = 0.852
```

In [8]:

```
fig, ax = plt.subplots(2, 2, figsize=(11, 7.5), sharex=True)
kd = 1000                                       # plot in $ thousand
panels = [
    (ax[0, 0], '(1) Empirical', [(var_1, 'C0', 'empirical')]),
    (ax[0, 1], '(2) Normal, EWMA moments', [(var_2, 'C1', 'normal')]),
    (ax[1, 0], '(3) GEV block maxima', [(gev[73]['var'], 'C2', 'n = 73, m =
                                        (gev[35]['var'], 'C4', 'n = 35, m =
    (ax[1, 1], '(4) GP excess losses, u = VaR$_{0.95}$', [(var_4, 'C3', 'GPD
]
for a_, title, curves in panels:
    for v, col, lab in curves:
        a_.plot(alphas, v / kd, color=col, lw=1.6, label=lab)
    a_.set_title(title, fontsize=10)
    a_.set_ylabel('VaR$_\\alpha$ (\\$ thousand)')
    a_.legend(fontsize=8)
    a_.grid(alpha=0.3)
for a_ in ax[1]:
    a_.set_xlabel('$\\alpha$')
```

```
fig.tight_layout()
plt.show()
```

